

High-Performance Sequential Quasi-Monte Carlo on Accelerators

Full chapter draft

Dissertation working draft

31 August 2026

Contents

1	Introduction	2
2	Algorithmic foundations: SMC, QMC and SQMC	2
2.1	State-space models and filtering	2
2.2	Sequential Monte Carlo	3
2.2.1	Algorithm 1: bootstrap particle filter	4
2.3	Quasi-Monte Carlo	4
2.3.1	Radical inverses and Halton points	5
2.3.2	Digital nets and Sobol points	5
2.3.3	Randomised QMC	5
2.4	Sequential quasi-Monte Carlo	5
2.4.1	Algorithm 2: Hilbert-ordered SQMC	6
2.5	Dimension and long-horizon behaviour	6
3	Why SQMC maps naturally onto accelerators	7
3.1	From statistical operations to accelerator primitives	7
3.2	Mapping from the graphics papers	8
3.3	Paper-derived implementation patterns	9
3.4	Parallelism, precision and reproducibility	10
4	A JAX implementation of SQMC	10
4.1	Software architecture	10
4.2	QMC generation	11
4.3	Hilbert sorting	11
4.4	Ordered inverse-CDF resampling	12
4.5	Rao-Blackwellised propagation	12
4.6	Algorithm 3: implemented RB-SQMC update	12
5	Empirical evaluation: accuracy, throughput and safety	13
5.1	Questions and comparisons	13
5.2	Accuracy and statistical efficiency	13
5.3	Throughput and speedup	13
5.4	Kernel-level breakdown	14
5.5	Dimension scaling	14

5.6	Long-horizon safety experiment	14
5.7	Figures and reporting order	15
6	Discussion	15
7	References	15

1 Introduction

Sequential Monte Carlo (SMC) is one of the most general computational tools for online inference in nonlinear and non-Gaussian state-space models. Its flexibility, however, comes with the ordinary Monte Carlo error scale $O_P(N^{-1/2})$: reducing error by a factor of two generally requires about four times as many particles. Sequential quasi-Monte Carlo (SQMC) replaces the independent uniforms used by a particle filter with randomized low-discrepancy point sets. It preserves the recursive structure of SMC while arranging the simulation so that the uniformity of a quasi-Monte Carlo design is carried through resampling and propagation [Gerber and Chopin \(2015\)](#).

The central argument of this chapter is that SQMC is both a statistical and a computationally natural method for accelerator-based filtering. Its principal operations are batched transforms, digital-net generation, reductions, space-filling-curve indexing, sorting and searching. These operations have regular array representations and can be compiled by JAX for a CPU, GPU or TPU. Related work in computer graphics provides useful engineering evidence: radical inverses and Sobol points have compact integer implementations, while Hilbert ordering has been used to distribute low-discrepancy samples over large parallel rendering workloads [Keller, Wachter and Binder \(2022\)](#); [Keller, Wachter and Binder \(2023\)](#).

The contribution developed here is the bridge between those two literatures. First, the SMC, QMC and SQMC recursions are stated in common notation, including complete algorithms. Second, each mathematical primitive is mapped to an accelerator operation and to its implementation in this repository. Third, the chapter defines an empirical protocol for separating statistical efficiency from hardware throughput. The distinction matters: a fast implementation of a poor estimator is not useful, while an estimator with a favourable asymptotic rate may still lose at practical particle counts if its sorting overhead is too large.

2 Algorithmic foundations: SMC, QMC and SQMC

2.1 State-space models and filtering

Let $X_t \in \mathcal{X} \subseteq \mathbb{R}^d$ be a latent Markov process and let Y_t be its observation. With initial density μ_0 , transition density f_t and observation density g_t , the model factorises as

$$p(x_{0:T}, y_{0:T}) = \mu_0(x_0) g_0(y_0 | x_0) \prod_{t=1}^T f_t(x_t | x_{t-1}) g_t(y_t | x_t).$$

The filtering distribution is $\pi_t(dx_t) = p(x_t | y_{0:t}) dx_t$. Its exact recursion consists of a prediction

$$\pi_{t|t-1}(dx_t) = \int f_t(x_t | x_{t-1}) \pi_{t-1}(dx_{t-1}) dx_{t-1}$$

followed by a correction

$$\pi_t(dx_t) = \frac{g_t(y_t | x_t) \pi_{t|t-1}(dx_t)}{\int g_t(y_t | z) \pi_{t|t-1}(dz)}.$$

Except in special cases, such as a linear Gaussian model, these integrals are not available in closed form. Particle methods replace each distribution by a finite weighted empirical measure.

2.2 Sequential Monte Carlo

At time t , an SMC approximation is

$$\pi_t^N(dx) = \sum_{n=1}^N W_t^n \delta_{X_t^n}(dx), \quad W_t^n = \frac{w_t^n}{\sum_{m=1}^N w_t^m}.$$

For a test function φ , the corresponding filtering estimate is

$$\pi_t^N(\varphi) = \sum_{n=1}^N W_t^n \varphi(X_t^n).$$

To describe a general auxiliary particle filter, let $m_t(x_{t-1}, dx_t)$ be a proposal kernel. After selecting an ancestor A_{t-1}^n with probability W_{t-1}^m , draw $X_t^n \sim m_t(X_{t-1}^{A_{t-1}^n}, \cdot)$. The incremental importance weight is

$$w_t^n = \frac{g_t(y_t | X_t^n) f_t(X_t^n | X_{t-1}^{A_{t-1}^n})}{m_t(X_t^n | X_{t-1}^{A_{t-1}^n})}.$$

For the bootstrap filter, $m_t = f_t$, so $w_t^n = g_t(y_t | X_t^n)$. Weight degeneracy may be monitored using the effective sample size

$$\text{ESS}_t = \frac{1}{\sum_{n=1}^N (W_t^n)^2},$$

with resampling triggered when ESS_t falls below a chosen fraction of N . When resampling occurs at every time step, a common estimate of the marginal likelihood is

$$\hat{p}(y_{0:T}) = \left(\frac{1}{N} \sum_{n=1}^N w_0^n \right) \prod_{t=1}^T \left(\frac{1}{N} \sum_{n=1}^N w_t^n \right).$$

2.2.1 Algorithm 1: bootstrap particle filter

Input: particle count N , observations $y_{0:T}$, initial simulator Γ_0 and transition simulator Γ_t .

1. Generate independent $U_0^n \sim \mathcal{U}([0, 1]^d)$ and set $X_0^n = \Gamma_0(U_0^n)$.
2. Set $w_0^n = g_0(y_0 \mid X_0^n)$ and normalise to obtain W_0^n .
3. For $t = 1, \dots, T$, perform the following four operations:
 - draw $A_{t-1}^n \sim \text{Categorical}(W_{t-1}^{1:N})$;
 - draw independent $V_t^n \sim \mathcal{U}([0, 1]^d)$;
 - propagate $X_t^n = \Gamma_t(X_{t-1}^{A_{t-1}^n}, V_t^n)$; and
 - set $w_t^n = g_t(y_t \mid X_t^n)$ and normalise.
4. Return $\{X_t^n, W_t^n\}_{n=1}^N$ for every required time.

Under standard regularity conditions and for fixed t , SMC estimates satisfy a central limit theorem with error $O_P(N^{-1/2})$. The constant may depend strongly on the model, proposal, resampling scheme and time horizon. SMC is therefore a linear-cost method per time step, but one whose accuracy grows only with the square root of computational effort.

2.3 Quasi-Monte Carlo

Consider the integral

$$I(\varphi) = \int_{[0,1]^s} \varphi(u) du,$$

$$\hat{I}_N(\varphi) = \frac{1}{N} \sum_{n=1}^N \varphi(u^n).$$

Monte Carlo chooses u^n independently. QMC instead chooses a deterministic point set that fills the unit cube evenly. For anchored boxes $[0, a) = \prod_{j=1}^s [0, a_j)$, its star discrepancy is

$$D_N^*(u^{1:N}) = \sup_{a \in [0,1]^s} \left| \frac{1}{N} \sum_{n=1}^N \mathbf{1}\{u^n \in [0, a)\} - \prod_{j=1}^s a_j \right|.$$

If φ has bounded variation in the Hardy-Krause sense, the Koksma-Hlawka inequality gives

$$|\hat{I}_N(\varphi) - I(\varphi)| \leq V_{\text{HK}}(\varphi) D_N^*(u^{1:N}).$$

For well-constructed sequences, $D_N^* = O\{N^{-1}(\log N)^s\}$. This is a worst-case deterministic bound, not a universal statement that every QMC estimator has exactly $O(N^{-1})$ error. It also exposes the dimension dependence: the nominal logarithmic factor can be severe when s is large, even though effective dimension is often more predictive than nominal dimension in applications.

2.3.1 Radical inverses and Halton points

Write the non-negative integer n in base b as $n = \sum_{k \geq 0} a_k(n)b^k$. The radical inverse reflects these digits across the radix point:

$$\phi_b(n) = \sum_{k \geq 0} a_k(n)b^{-k-1}.$$

The s -dimensional Halton sequence uses distinct prime bases,

$$u^n = (\phi_{b_1}(n), \dots, \phi_{b_s}(n)).$$

Digit permutations replace a_k by $\sigma_{b,k}(a_k)$ and can reduce unwanted correlations between coordinates. This is the idea behind generalised and scrambled Halton constructions [Faure and Lemieux \(2010\)](#).

2.3.2 Digital nets and Sobol points

Sobol sequences are digital (t, s) -sequences in base two. If $n = \sum_{k=0}^{m-1} a_k(n)2^k$ and C_j is the binary generator matrix for coordinate j , then its output digits satisfy

$$\begin{pmatrix} z_{j,1} \\ \vdots \\ z_{j,m} \end{pmatrix} = C_j \begin{pmatrix} a_0(n) \\ \vdots \\ a_{m-1}(n) \end{pmatrix} \pmod{2}, \quad u_j^n = \sum_{r=1}^m z_{j,r}2^{-r}.$$

In software, multiplication over $\mathbb{F}_2$ becomes a sequence of masks and bitwise exclusive-or operations. This integer representation is important both for speed and for reproducibility.

2.3.3 Randomised QMC

Randomised QMC (RQMC) applies a randomisation that preserves low discrepancy while making each point marginally uniform. If $\tilde{u}^n$ denotes a scrambled point, then

$$\mathbb{E} \left[\frac{1}{N} \sum_{n=1}^N \varphi(\tilde{u}^n) \right] = I(\varphi).$$

Independent scrambles provide replicates for variance estimation. A left linear matrix scramble followed by a digital shift is especially convenient for Sobol points because both stages are performed over $\mathbb{F}_2$.

2.4 Sequential quasi-Monte Carlo

Directly replacing the random uniforms in SMC with QMC points is not enough. After resampling, an unordered categorical distribution disconnects nearby QMC points from nearby particles. SQMC restores a one-dimensional ordering of the particle cloud before applying the inverse transform.

Let $\psi : \mathcal{X} \rightarrow [0, 1)^d$ be a component-wise monotone mapping and let $h : [0, 1)^d \rightarrow [0, 1)$ denote a pseudo-inverse of the Hilbert space-filling curve. Define the scalar key

$$z_{t-1}^n = h\{\psi(X_{t-1}^n)\}.$$

Let σ sort particles by z_{t-1}^n , and let τ sort the first coordinates of an RQMC point set

$$(u_t^n, v_t^n) \in [0, 1) \times [0, 1)^d.$$

For the ordered weights, define

$$C_j = \sum_{k=1}^j W_{t-1}^{\sigma(k)}, \quad a_t^n = \min\{j : C_j \geq u_t^{\tau(n)}\}.$$

The propagated particle is then

$$X_t^n = \Gamma_t(X_{t-1}^{\sigma(a_t^n)}, v_t^{\tau(n)}).$$

The pairing of $u_t^{\tau(n)}$ and $v_t^{\tau(n)}$ must be retained: the first coordinate chooses the ancestor and the remaining coordinates propagate that same point. Breaking this pairing destroys the joint low-discrepancy design.

2.4.1 Algorithm 2: Hilbert-ordered SQMC

Input: N , $y_{0:T}$, RQMC generator, Γ_0 , Γ_t , and potential functions G_t .

1. Generate $u_0^{1:N} \subset [0, 1)^d$ and set $X_0^n = \Gamma_0(u_0^n)$.
2. Compute $w_0^n = G_0(X_0^n)$ and normalise to W_0^n .
3. For $t = 1, \dots, T$:
 - a. Generate $(u_t^n, v_t^n)_{n=1}^N \subset [0, 1)^{d+1}$.
 - b. Compute $z_{t-1}^n = h\{\psi(X_{t-1}^n)\}$ and obtain the particle permutation σ that sorts $z_{t-1}^{1:N}$.
 - c. Obtain τ by sorting $u_t^{1:N}$ while keeping each v_t^n paired with its first coordinate.
 - d. Form cumulative ordered weights $C_j = \sum_{k \leq j} W_{t-1}^{\sigma(k)}$.
 - e. Set $a_t^n = \min\{j : C_j \geq u_t^{\tau(n)}\}$.
 - f. Propagate $X_t^n = \Gamma_t(X_{t-1}^{\sigma(a_t^n)}, v_t^{\tau(n)})$.
 - g. Compute $w_t^n = G_t(X_{t-1}^{\sigma(a_t^n)}, X_t^n)$ and normalise.
4. Return the weighted particle systems.

The two sorts make the generic per-step complexity $O(N \log N)$, compared with $O(N)$ for a conventional particle filter with a linear-time resampler. Gerber and Chopin (2015) establish consistency and RQMC error results that are asymptotically better than ordinary Monte Carlo under their stated conditions. The gain is problem dependent; it should be measured rather than assumed for a particular model and particle count.

2.5 Dimension and long-horizon behaviour

The construction uses a $(d + 1)$ -dimensional point set at each non-initial time: one coordinate for resampling and d for propagation. As d grows, both the discrepancy bound and the cost of the Hilbert index deteriorate. Dimension reduction is therefore part of the algorithmic design, not

merely an implementation optimisation. Rao-Blackwellisation, conditional simulation, Brownian-bridge constructions and principal-component constructions can all move variation into fewer QMC coordinates. Chapter 2 uses this principle by integrating a conditionally Gaussian component analytically and applying SQMC only to the lower-dimensional sampled component.

A separate argument concerns stability over long horizons. For the specific one-dimensional linear Gaussian example studied by Gerber (2026), a conventional particle filter has errors above a fixed threshold infinitely often, almost surely, for every fixed N . In the same example, the analysed SQMC construction satisfies

$$\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\|_K = 0 \quad \text{almost surely,}$$

where $\|\cdot\|_K$ is Kolmogorov distance. This is a compelling motivation for a long-horizon experiment, but it is not a universal theorem for all state-space models. The empirical section therefore treats it as a targeted illustration of the published result.

3 Why SQMC maps naturally onto accelerators

3.1 From statistical operations to accelerator primitives

An accelerator executes large collections of regular operations most efficiently when data remain on device and control flow is static. SQMC is not embarrassingly parallel end to end: reductions, sorting and inverse-CDF search are collective operations. Nevertheless, its complete step can be expressed using mature parallel primitives.

SQMC operation	Mathematical role	Accelerator primitive	Main cost or risk
RQMC generation	Produce (u_t^n, v_t^n)	integer masks, shifts, XOR, batched scans	direction-table access and compilation
State mapping	Compute $\psi(X^n)$	elementwise standardisation and sigmoid	outliers and batch dependence
Hilbert index	Compute $h\{\psi(X^n)\}$	bit transpose, Gray code, fixed scan	fewer bits per coordinate as d grows
Particle ordering	Obtain σ	device sort	$O(N \log N)$ and memory traffic
Weight normalisation	Obtain W^n	log-sum-exp reduction	precision and overflow
Ordered CDF	Form C_j	parallel prefix scan	collective synchronisation
Ancestor lookup	Invert C at sorted u^n	sort plus search	branch and access regularity
Propagation	Apply Γ_t	vectorised linear algebra	model-specific arithmetic intensity
Weighting	Evaluate G_t	batched likelihood kernel	model-specific bottleneck

This table corrects a tempting but inaccurate claim: Hilbert sorting does not replace the cumulative-weight scan. Standard SQMC still constructs C_j and inverts it. Hilbert ordering solves a dif-

ferent problem - it converts the multidimensional particle system into a locality-preserving one-dimensional order so that ordered resampling can retain QMC regularity. Both sort and scan must therefore be included in any honest timing breakdown.

3.2 Mapping from the graphics papers

The graphics papers provide implementation patterns rather than a ready-made SQMC filter. The mapping is as follows.

Paper idea	Paper-level object	SQMC analogue	Repository implementation
Integer radical inversion	$\phi_b(i)$	Halton coordinate generation	<code>Halton._radical_inverse</code>
Digital scrambling	$\phi_{b,\sigma}(i)$	RQMC randomisation	Halton permutations; Sobol LMS plus shift
Sobol generator matrices	$C_j a(i)$ over $\mathbb{F}_2$	$(d+1)$ -dimensional RQMC points	<code>Sobol.sample</code> and <code>_apply_lms</code>
Hilbert pixel enumeration	pixel $\leftrightarrow$ curve index	particle $\mapsto$ scalar ordering key	<code>Hilbert_to_int</code> and <code>hilbert_sort</code>
Contiguous point blocks	nearby pixels receive nearby indices	nearby particles become neighbours before resampling	<code>sort_idx</code> in <code>hilbert_resample</code>
Massively parallel execution	independent paths/pixels	batched particles and likelihoods	<code>jax.vmap</code> , <code>jax.lax.scan</code> and <code>jax.jit</code>

Keller, Wachter and Binder (2022) enumerate pixels along a Hilbert curve and assign contiguous pieces of a low-discrepancy sequence to spatially adjacent pixels. Their progressive construction can be written as

$$\mathcal{P}_j = \{x_{\ell N+j} : \ell \in \mathbb{N}_0\},$$

where j is the Hilbert index of a pixel and N is the number of pixels. SQMC does not partition samples in exactly this way, but uses the same structural idea: a multidimensional state is assigned a scalar locality-preserving index, after which consecutive low-discrepancy coordinates act on consecutive locations in that order.

3.3 Paper-derived implementation patterns

The following code is a compact adaptation of the integer-first radical inverse pattern in Keller, Wachter and Binder (2023), not a verbatim listing. It accumulates reversed digits as an integer and converts to floating point only once.

```
float radical_inverse(uint32_t index, uint32_t base) {
    uint32_t reversed = 0;
    uint32_t scale = 1;
    do {
        reversed = reversed * base + index % base;
        index /= base;
        scale *= base;
    } while (index != 0);
    return float(reversed) / float(scale);
}
```

The corresponding JAX pattern in this repository replaces the scalar loop by a fixed-length compiled scan over an entire vector of indices:

```
def step(state, _):
    remaining, value, factor = state
    digit = remaining % base_i
    value = value + digit.astype(dtype) * factor
    return (remaining // base_i, value, factor / base_f), None

state, _ = jax.lax.scan(step, initial_state, None,
                        length=num_digits)
```

The paper’s Sobol implementation reads several generator-matrix columns at a time and conditionally XORs them into an integer accumulator. Abstracting away the CUDA vector type, its core operation is

```
for (uint32_t bit = 0; index != 0; ++bit, index >>= 1) {
    if (index & 1u) value ^= direction[dimension][bit];
}
value ^= digital_shift;
```

In JAX, direction integers and scramble masks are arrays, while bitwise XOR and population parity implement the same arithmetic over $\mathbb{F}_2$. This is a genuine implementation correspondence: the storage layout differs, but the algebra is the same.

Hilbert ordering is a different kind of mapping. The rendering paper assumes a two-dimensional pixel grid whose Hilbert index can be computed directly. SQMC must accept arbitrary particle dimension and continuous coordinates. The local implementation therefore performs three additional stages:

1. standardise every coordinate and apply a logistic map into $(0, 1)$;
2. quantise each coordinate on a finite binary grid;
3. decode oriented Gray-code chunks and pack them into one 62-bit key.

An abridged version of the recursion is

```
def visit_child(orientation, chunk):
    start, end = orientation
    step = gray_decode_travel(start, end, mask, chunk)
    child = child_start_end(start, end, mask, step)
    return child, step

_, chunks = jax.lax.scan(
    visit_child, initial_orientation, coordinate_chunks)
```

The implementation is thus inspired by the papers’ integer and space-filling-curve principles; the returned chunks are packed into the final integer key. It is not copied from a paper-specific pixel routine. That distinction is important for attribution and for technical accuracy.

3.4 Parallelism, precision and reproducibility

Point generation, coordinate mapping, propagation and weighting are parallel over particles. Sorting, reductions and scans are parallel collective operations with more communication. The total time for one SQMC update may be decomposed as

$$t_{\text{step}} = t_{\text{qmc}} + t_{\text{map}} + t_{\text{Hilbert}} + t_{\text{sort}} + t_{\text{scan}} + t_{\text{search}} + t_{\text{prop}} + t_{\text{weight}}.$$

This decomposition is more informative than labelling the whole method “embarrassingly parallel”. It also determines the benchmark instrumentation in the final section.

Integer-first QMC construction improves reproducibility, but floating-point normalisation and cumulative sums remain sensitive to precision. Parallel resampling studies report that cumulative weights can become numerically problematic at large N in single precision [Murray, Lee and Jacob \(2016\)](#). The present implementation enables 64-bit JAX arithmetic, normalises weights in log space and clips QMC endpoints before applying a Gaussian quantile. Lower precision may improve throughput, but it should be evaluated as a separate accuracy-performance trade-off rather than enabled silently.

4 A JAX implementation of SQMC

4.1 Software architecture

The implementation separates reusable SQMC primitives from the application filter:

- `sqmc/qmc/qmc.py` implements Halton and Sobol generators;
- `sqmc/hilbert_sort/hilbert_sort.py` implements Hilbert keys and sorting;
- `rbsqmc/src/model/model_rbsqmc.py` combines RQMC generation, ordered resampling, Rao-Blackwellised propagation and observation weighting.

JAX traces these array programs and lowers them through XLA. The same source therefore targets CPU or accelerator backends, although identical source does not guarantee identical kernel fusion or numerical results across devices.

4.2 QMC generation

For Halton points, the code vectorises over all requested indices and uses a fixed `lax.scan` over base- b digits. Optional per-position permutations produce a randomised Halton design. For Sobol points, checked-in Joe-Kuo direction integers are scrambled using unit lower-triangular binary matrices, followed by a digital shift. For a binary input vector a and scramble matrix L , the construction is

$$z_j = L_j C_j a + s_j \pmod{2}.$$

The matrix is unit lower triangular and hence invertible over $\mathbb{F}_2$; the scramble changes the realised net without collapsing it. In the filter, each valid match update calls

```
rqmc_points = generate_rqmc_points(
    key=sobol_key,
    n=n_particles,
    d=1 + effective_dimension,
)
```

The first column is reserved for resampling. The remaining columns drive the conditional state transform.

4.3 Hilbert sorting

For $d \geq 2$, one 62-bit key is shared across dimensions. The resolution per coordinate is

$$b_d = \left\lfloor \frac{62}{d} \right\rfloor, \quad q_j = \min \{ 2^{b_d} - 1, \lfloor 2^{b_d} \psi_j(x_j) \rfloor \}.$$

Coordinate bits are transposed into level-wise chunks. A scan tracks the start and end corners of the current Hilbert sub-cube, decodes each chunk in that orientation and packs the resulting steps into a scalar unsigned integer. The particle permutation is then a stable `argsort` of these keys. In one dimension the implementation correctly reduces to an ordinary stable sort.

The component-wise map used locally is batch standardisation followed by a logistic transform,

$$\psi_j(x_j^n) = \left[1 + \exp \left(-\frac{x_j^n - \bar{x}_j}{s_j} \right) \right]^{-1}.$$

Constant coordinates use a safe scale and map to the middle of the interval. Because $\bar{x}_j$ and s_j depend on the current particle cloud, this is a pragmatic finite-sample map rather than a fixed theoretical bijection. Its effect should be checked empirically against fixed marginal transforms.

4.4 Ordered inverse-CDF resampling

The repository implementation follows the SQMC construction directly:

```
sort_idx = hilbert_sort(x_flat)
w = jnp.exp(log_weights - logsumexp(log_weights))
cdf = jnp.cumsum(w[sort_idx])

qmc_order = jnp.argsort(rqmc_points[:, 0], stable=True)
rqmc_sorted = rqmc_points[qmc_order]
positions = jnp.searchsorted(cdf, rqmc_sorted[:, 0])
ancestors = sort_idx[jnp.minimum(positions, n_particles - 1)]
v_t = rqmc_sorted[:, 1:]
```

This listing makes three correctness conditions visible. Weights are reordered by the same permutation as the particles; complete QMC rows are reordered so that u and v remain paired; and indices returned in Hilbert order are mapped back to the original particle array.

4.5 Rao-Blackwellised propagation

For the football model in Chapter 2, only the two teams playing a match need new sampled coordinates. Each team has a two-dimensional latent state, so the effective sampled dimension is four and every non-initial RQMC point has five coordinates. Let $\mu_O^n \in \mathbb{R}^4$ be the predicted state of the two observed teams and let $\Sigma_{OO} = \Gamma_{OO} \otimes B$. With $LL^\top = \Sigma_{OO}$ and $z_t^n = \Phi^{-1}(v_t^n)$, propagation is

$$x_{O,t}^n = \mu_{O,t}^n + L_t z_t^n.$$

The remaining conditionally Gaussian team states are updated with the Kalman gain K_t :

$$x_t^n = \mu_t^n + K_t(x_{O,t}^n - \mu_{O,t}^n),$$

with the sampled observed block inserted exactly. The code clips v_t^n away from zero and one before applying Φ^{-1} to avoid infinite floating-point quantiles.

4.6 Algorithm 3: implemented RB-SQMC update

For each valid match at time t :

1. Split the random key and generate N independently scrambled Sobol points in $[0, 1)^5$.
2. Extract and flatten the four sampled coordinates from every particle.
3. Hilbert-sort these coordinates and reorder the normalised weights.
4. Sort complete Sobol rows by their first coordinate.
5. Construct the ordered weight CDF and select ancestors with `searchsorted`.
6. Propagate the four observed coordinates using the Gaussian inverse transform.
7. Recover the complementary coordinates with the Kalman conditional update.
8. Evaluate the bivariate Poisson log-potential for all particles.
9. Update the log normalising constant with `logsumexp` and normalise the new log-weights.

The outer date loop and inner match loop are both expressed with `jax.lax.scan`, while particle likelihoods use `jax.vmap`. Once compiled, the entire update remains on the selected device.

5 Empirical evaluation: accuracy, throughput and safety

5.1 Questions and comparisons

The evaluation should answer three questions.

1. Does SQMC reduce filtering error relative to SMC at the same N ?
2. Does accelerator execution offset the extra sorting cost and reduce time-to-accuracy?
3. Which kernels dominate runtime as N and effective dimension change?

The principal comparison contains SMC-CPU, SQMC-CPU and SQMC-GPU. If a GPU SMC baseline is available, it should also be reported; otherwise hardware and algorithm effects are partially confounded. CPU and GPU SQMC must use the same precision, model, data, particle counts and randomisation scheme wherever possible.

5.2 Accuracy and statistical efficiency

For a latent reference trajectory $x_{1:T}^*$, define filtering RMSE

$$\text{RMSE}(N) = \left[\frac{1}{TR} \sum_{r=1}^R \sum_{t=1}^T \|\hat{x}_{t,r}^N - x_t^*\|_2^2 \right]^{1/2},$$

where R is the number of independent SMC runs or independent RQMC scrambles. When the exact state is unavailable, use a high-particle reference filter and report that substitution. A log-log regression

$$\log \text{RMSE}(N) = \alpha - \beta \log N + \varepsilon_N$$

provides a descriptive empirical slope, but it should not be presented as a proof of an asymptotic rate.

The headline measure is time-to-accuracy,

$$T_\epsilon = \inf\{t_{\text{wall}} : \text{RMSE}(t_{\text{wall}}) \leq \epsilon\}.$$

Plotting error against elapsed time combines estimator quality and hardware throughput in the quantity that matters to a user.

5.3 Throughput and speedup

After a separate compilation warm-up, time repeated synchronised executions. JAX dispatch is asynchronous, so every timed call must finish with `jax.block_until_ready`. For steady-state runtime $t_{N,T}$, report

$$\text{throughput}(N) = \frac{NT}{t_{N,T}}, \quad S(N) = \frac{t_{\text{CPU}}(N)}{t_{\text{GPU}}(N)}.$$

The report should include median, interquartile range and number of repeats, as well as CPU model, accelerator model, memory, JAX/JAXLIB versions, backend, precision and compiler warm-up policy. Compilation time should be reported separately rather than hidden or mixed into steady-state measurements.

5.4 Kernel-level breakdown

Instrument RQMC generation, state mapping, Hilbert-key construction, sorting, CDF scan, ancestor lookup, propagation and weighting. Kernel fusion may make isolated timings differ from their contribution to the end-to-end program, so report both the decomposed microbenchmark and the complete filtered step. The useful diagnostic is the fraction

$$r_k(N) = \frac{t_k(N)}{t_{\text{step}}(N)}$$

for each component k . In particular, separating Hilbert-key calculation from sorting, and sorting from the cumulative scan, tests the hardware narrative directly.

5.5 Dimension scaling

Repeat the synthetic benchmark for effective dimensions $d \in \{1, 2, 4, 8, 16\}$ where the model permits. Record runtime, discrepancy or error, and bits per Hilbert coordinate $\lfloor 62/d \rfloor$. This experiment distinguishes two effects that would otherwise be mixed: statistical loss from increasing QMC dimension and representational loss from a coarser packed Hilbert key.

5.6 Long-horizon safety experiment

Use the model analysed by Gerber (2026),

$$Y_t = X_t + c^{-1/2}Z_t, \quad X_{t+1} = \rho X_t + \sigma W_{t+1},$$

with $y_t = 0$ and independent standard Gaussian Z_t, W_t . For each method and particle count, compute the Kolmogorov distance to the analytic Gaussian filter,

$$D_t^N = \sup_{x \in \mathbb{R}} |\widehat{F}_t^N(x) - F_t(x)|.$$

Plot D_t^N over a long horizon and also report $\max_{1 \leq t \leq T} D_t^N$ across independent replicates. This finite experiment cannot demonstrate an “infinitely often” almost-sure statement. Its purpose is narrower: to illustrate whether the empirical behaviour at finite T is consistent with the mechanism proved for the paper’s model.

5.7 Figures and reporting order

The final empirical section should present:

1. filtering error against particle count;
2. filtering error against wall-clock time, with a target ϵ line;
3. particles per second against N ;
4. CPU-to-GPU speedup against N ;
5. the per-step timing breakdown;
6. dimension scaling; and
7. the long-horizon Kolmogorov-distance experiment.

No numerical result is asserted in this draft because the full controlled benchmark has not yet been inserted. This avoids turning an expected GPU advantage into a reported finding before the experiment has been run.

6 Discussion

SQMC introduces structure where an ordinary particle filter uses independent randomness. Statistically, this structure can reduce error by transporting a low-discrepancy design through ordered resampling and deterministic propagation. Computationally, it yields an array program dominated by integer point generation, transforms, sorting, scans and batched model evaluation. Those are plausible accelerator workloads, but not all have the same scaling: propagation and weighting are particle-parallel, whereas sorting and scanning require device-wide communication.

The graphics literature clarifies why the basic ingredients are practical. Radical inversion and Sobol construction can be implemented primarily with integer arithmetic, and Hilbert indices turn spatial locality into a scalar order without a large lookup table. The repository generalises those ideas to continuous, multidimensional particle states and composes them with JAX’s compiled transformations. The correspondence is conceptual and algebraic; it is not a claim that a rendering algorithm itself is an SQMC resampler.

Three limitations should remain visible. First, generic SQMC has $O(N \log N)$ sorting cost. Second, increasing effective dimension weakens QMC uniformity and reduces the resolution available to each coordinate in a packed Hilbert key. Third, numerical precision remains relevant because standard SQMC still constructs a cumulative distribution of floating-point weights. These limitations motivate Rao-Blackwellisation, explicit precision experiments and the kernel-level benchmark rather than weaken the case for the method.

The empirical criterion is ultimately time-to-accuracy. If the GPU reduces the cost per SQMC step sufficiently, and if SQMC needs fewer particles to reach a target error, the two gains multiply. The evaluation proposed above is designed to determine whether that combined advantage is realised for the filtering problem studied in this dissertation.

7 References

Chopin, N. and Gerber, M. (2017). Sequential quasi-Monte Carlo: introduction for non-experts, dimension reduction, application to partly observed diffusion processes. *Monte Carlo and Quasi-Monte Carlo Methods 2016*, 109-139. doi:[10.1007/978-3-319-91436-7_6](https://doi.org/10.1007/978-3-319-91436-7_6).

- Chopin, N. and Papaspiliopoulos, O. (2020). *An Introduction to Sequential Monte Carlo*. Springer. doi:10.1007/978-3-030-47845-2.
- Faure, H. and Lemieux, C. (2010). Generalized Halton sequences in 2008: a comparative study. *Monte Carlo Methods and Applications*, 16(3-4), 229-253. doi:10.1515/MCMA.2010.008.
- Gerber, M. and Chopin, N. (2015). Sequential quasi-Monte Carlo. *Journal of the Royal Statistical Society: Series B*, 77(3), 509-579. doi:10.1111/rssb.12084.
- Gerber, M. (2026). Safety of particle filters: some results on the time evolution of particle filter estimates. *Electronic Journal of Probability*, 31. doi:10.1214/26-EJP1581.
- Keller, A., Wachter, C. and Binder, N. (2022). Rendering along the Hilbert curve. arXiv:2207.05415. [arXiv record](#).
- Keller, A., Wachter, C. and Binder, N. (2023). Quasi-Monte Carlo algorithms (not only) for graphics software. arXiv:2307.15584. [arXiv record](#).
- L'Ecuyer, P., Lecot, C. and Tuffin, B. (2008). A randomized quasi-Monte Carlo simulation method for Markov chains. *Operations Research*, 56(4), 958-975. doi:10.1287/opre.1080.0556.
- Murray, L. M., Lee, A. and Jacob, P. E. (2016). Parallel resampling in the particle filter. *Statistics and Computing*, 26, 789-805. doi:10.1007/s11222-014-9545-3.